Hw3 : Fairness & Bias in AI

Name : Omkumar M. Patel

Student ID: 222110936

Baseline

Libraries & Dependencies

```
#run this once, start of each session.
!pip install ucimlrepo
```

```
Collecting ucimlrepo
  Downloading ucimlrepo-0.0.7-py3-none-any.whl.metadata (5.5 kB)
Requirement already satisfied: pandas>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from ucimlrepo) (2.2.2)
Requirement already satisfied: certifi>=2020.12.5 in /usr/local/lib/python3.12/dist-packages (from ucimlrepo) (2026.1.4)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->ucimlrepo) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->ucimlrepo) (
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->ucimlrepo) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->ucimlrepo) (2025.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas>=1.0.0->uc
Downloading ucimlrepo-0.0.7-py3-none-any.whl (8.0 kB)
Installing collected packages: ucimlrepo
Successfully installed ucimlrepo-0.0.7
```

```
# import pandas as pd
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from ucimlrepo import fetch_ucirepo
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
import torch.optim as optim
import os
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset
from torchvision import datasets, transforms
import itertools
```

Understanding the Dataset

```
# fetch dataset
adult = fetch_ucirepo(id=2)

# data (as pandas dataframes)
X = adult.data.features
y = adult.data.targets
```

```
adult.variables
```

	name	role	type	demographic	description	units	missing_values
0	age	Feature	Integer	Age	N/A	None	no
1	workclass	Feature	Categorical	Income	Private, Self-emp-not-inc, Self-emp-inc, Feder...	None	yes
2	fnlwgt	Feature	Integer	None	None	None	no
3	education	Feature	Categorical	Education Level	Bachelors, Some-college, 11th, HS-grad, Prof-...	None	no
4	education-num	Feature	Integer	Education Level	None	None	no
5	marital-status	Feature	Categorical	Other	Married-civ-spouse, Divorced, Never-married, S...	None	no
6	occupation	Feature	Categorical	Other	Tech-support, Craft-repair, Other-service, Sal...	None	yes
7	relationship	Feature	Categorical	Other	Wife, Own-child, Husband, Not-in-family, Other...	None	no
8	race	Feature	Categorical	Race	White, Asian-Pac-Islander, Amer-Indian-Eskimo,...	None	no
9	sex	Feature	Binary	Sex	Female, Male.	None	no
10	capital-gain	Feature	Integer	None	None	None	no
11	capital-loss	Feature	Integer	None	None	None	no
12	hours-per-week	Feature	Integer	None	None	None	no
13	native-country	Feature	Categorical	Other	United-States, Cambodia, England, Puerto-Rico,...	None	yes
14	income	Target	Binary	Income	>50K, <=50K.	None	no

```
adult.metadata
```

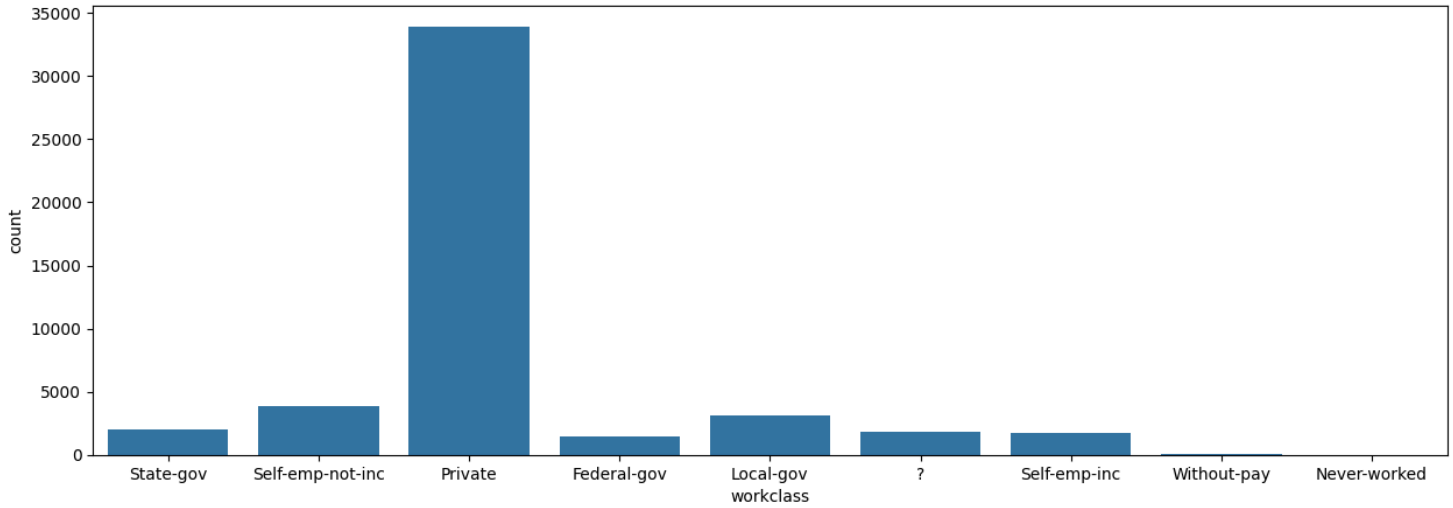
Show hidden output

```
df1 = pd.DataFrame(y)
df = pd.DataFrame(X)
df = pd.concat([df, df1], axis=1)
df.head(10)
```

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relationship	race	sex	capital-gain	capital-loss	hours-per-week	native-country
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	40	United States
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	13	United States
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	40	United States
3	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	0	40	United States
4	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	0	40	Cuba
5	37	Private	284582	Masters	14	Married-civ-spouse	Exec-managerial	Wife	White	Female	0	0	40	United States
6	49	Private	160187	9th	5	Married-spouse-absent	Other-service	Not-in-family	Black	Female	0	0	16	Jamaica
7	52	Self-emp-not-inc	209642	HS-grad	9	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	45	United States
8	31	Private	45781	Masters	14	Never-married	Prof-specialty	Not-in-family	White	Female	14084	0	50	United States
9	42	Private	159449	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	5178	0	40	United States

Visualizations

```
plt.figure(figsize=(15,5))
sns.countplot(x='workclass', data=df)
plt.show()
```




```
#contingency table of workclass and income
crosstab_df = pd.crosstab(df['workclass'], df['income'], normalize='index')

plt.figure(figsize=(12, 8))
sns.heatmap(crosstab_df, annot=True, fmt=".2%", cmap="YlGnBu", linewidths=.5)
plt.title('Proportion of Income Levels by Workclass')
plt.xlabel('Income')
plt.ylabel('Workclass')
plt.show()
```

Show hidden output

```
df['income'].value_counts()
```

income	count
<=50K	24720
<=50K.	12435
>50K	7841
>50K.	3846

dtype: int64

Fixing Typo and Assigning Binary Labels.

```
df['income'] = df['income'].replace({'<=50K.': 0, '<=50K':0, '>50K.': 1, '>50K':1 })
df['income'].value_counts()
```

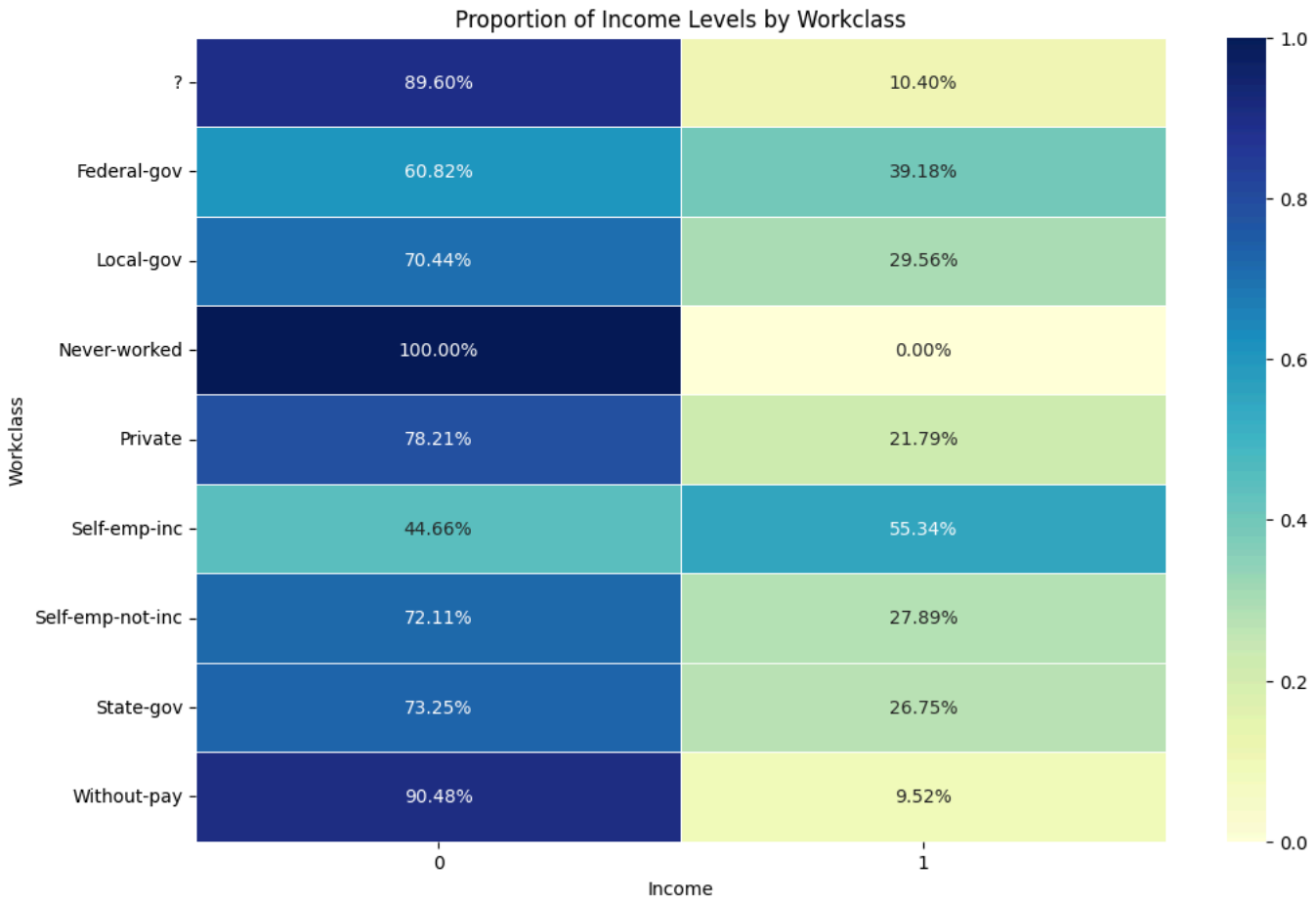
```
/tmp/ipython-input-224/1855584782.py:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a fu
  df['income'] = df['income'].replace({'<=50K.': 0, '<=50K':0, '>50K.': 1, '>50K':1 })
```

income	count
0	37155
1	11687

dtype: int64

```
#contingency table of workclass and income
crosstab_df = pd.crosstab(df['workclass'], df['income'], normalize='index')

plt.figure(figsize=(12, 8))
sns.heatmap(crosstab_df, annot=True, fmt=".2%", cmap="YlGnBu", linewidths=.5)
plt.title('Proportion of Income Levels by Workclass')
```

```
plt.xlabel('Income')
plt.ylabel('Workclass')
```




```
Start coding or generate with AI.
```

Preprocessing

```
# dropping null values
df = df.dropna(axis=0).reset_index(drop=True)
X = df.drop('income', axis=1)
y = df['income'].astype(int)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
```

```
X_train_raw = X_train.copy()
X_test_raw = X_test.copy()

def encode_sex_binary(series):
    mapped = series.astype(str).str.strip().str.lower().map({'male': 1, 'female': 0})
    if mapped.isna().any():
        bad = series[mapped.isna()].astype(str).unique()[:10]
        raise ValueError(f'Unexpected values in sex column: {bad}')
    return mapped.astype(int).values

# sensitive attributes extracted from RAW (string) values before encoding
sex_train = encode_sex_binary(X_train_raw['sex'])
sex_test = encode_sex_binary(X_test_raw['sex'])
# race_test = X_test["race"].astype(str).str.strip().values

X_train = X_train_raw.copy()
X_test = X_test_raw.copy()

cat_col = X_train.select_dtypes(include=['object', 'category']).columns
print(cat_col)
print('sex_train counts:', pd.Series(sex_train).value_counts().to_dict())
print('sex_test counts :', pd.Series(sex_test).value_counts().to_dict())

```

```
Index(['workclass', 'education', 'marital-status', 'occupation',
       'relationship', 'race', 'sex', 'native-country'],
      dtype='object')
sex_train counts: {1: 22356, 0: 10978}
sex_test counts : {1: 9581, 0: 4706}
```

```
#Chaning Categorical Features to numerical
le = LabelEncoder()

encoders = {}
for col in cat_col:

    train_vals = X_train[col].astype(str).values
    # add UNK to prevent crashes if test contains unseen category
    le.fit(np.concatenate([train_vals, np.array(["UNK"], dtype=object)]))
    encoders[col] = le

    X_train[col] = le.transform(train_vals)

    test_vals = X_test[col].astype(str).values
    known = np.isin(test_vals, le.classes_)
    test_safe = np.where(known, test_vals, "UNK")
    X_test[col] = le.transform(test_safe)

```

```
# Normalizing the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train.values.astype(np.float32)).astype(np.float32)
X_test_scaled  = scaler.transform(X_test.values.astype(np.float32)).astype(np.float32)

y_train_scaled = y_train.values.astype(np.float32).reshape(-1, 1)
y_test_scaled  = y_test.values.astype(np.float32).reshape(-1, 1)

print("X_train:", X_train_scaled.shape, "y_train:", y_train_scaled.shape,"\n","X_test :", X_test_scaled.shape,  "y_test :", y
print("sex_test:", sex_test.shape,'sex_train:', sex_train.shape,)
print("Categorical col enc:", cat_col)
```

```
X_train: (33334, 14) y_train: (33334, 1)
 X_test : (14287, 14) y_test : (14287, 1)
sex_test: (14287,) sex_train: (33334,)
Categorical col enc: Index(['workclass', 'education', 'marital-status', 'occupation',
       'relationship', 'race', 'sex', 'native-country'],
      dtype='object')
```

```
# print(X_train_scaled)
# print(X_train.head())
print(sex_train)
```

```
[1 0 0 ... 1 0 1]
```

Start coding or generate with AI.

Start coding or generate with AI.

Training

```
def make_loader(X_np, y_np, batch_size=32, shuffle=False, sample_weights=None):
    X_t = torch.tensor(X_np, dtype=torch.float32)
    y_t = torch.tensor(y_np, dtype=torch.float32)
    # ds = TensorDataset(X_t, y_t)

    if sample_weights is None:
        ds = TensorDataset(X_t, y_t)
    else:
        w_t = torch.tensor(sample_weights.reshape(-1, 1), dtype=torch.float32)
        ds = TensorDataset(X_t, y_t, w_t)

    return DataLoader(ds, batch_size=batch_size, shuffle=shuffle)

train_loader = make_loader(X_train_scaled, y_train_scaled, batch_size=32, shuffle=True)
test_loader  = make_loader(X_test_scaled,  y_test_scaled,  batch_size=1024, shuffle=False)

```

```python
class MLP(nn.Module):
    # 2 hidden layers (64 each), ReLU; output 1 neuron with Sigmoid
    def __init__(self, input_dim: int):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, 64)
        self.fc2 = nn.Linear(64, 64)
        self.fc3 = nn.Linear(64, 1)
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.sigmoid(self.fc3(x))
        return x

input_size = X_train_scaled.shape[1]
model = MLP(input_size)
print(model)

def probs_to_pred(probs, threshold=0.5):
    return (probs > threshold).astype(int)

@torch.no_grad()
def predict_proba_full(model, X_np):
    model.eval()
    X = torch.tensor(X_np, dtype=torch.float32)
    probs = model(X).detach().cpu().numpy().reshape(-1)
    return probs

def confusion_counts(y_true, y_pred):
    y_true = y_true.astype(int)
    y_pred = y_pred.astype(int)
    tp = np.sum((y_true == 1) & (y_pred == 1))
    fn = np.sum((y_true == 1) & (y_pred == 0))
    fp = np.sum((y_true == 0) & (y_pred == 1))
    tn = np.sum((y_true == 0) & (y_pred == 0))
    return tp, fn, fp, tn

def tpr(y_true, y_pred):
    tp, fn, _, _ = confusion_counts(y_true, y_pred)
    denom = tp + fn
    return float(tp / denom) if denom > 0 else float("nan")

def fpr(y_true, y_pred):
    _, _, fp, tn = confusion_counts(y_true, y_pred)
    denom = fp + tn
    return float(fp / denom) if denom > 0 else float("nan")

def acc(y_true, y_pred):
  accuracy = float(np.mean(y_true.astype(int) == y_pred.astype(int)))
  return accuracy

def equal_opportunity_gap(y_true, y_pred, sex_binary):
    male = (sex_binary == 1)
    female = (sex_binary == 0)
    tpr_m = tpr(y_true[male], y_pred[male])
    tpr_f = tpr(y_true[female], y_pred[female])
    gap = abs(tpr_m - tpr_f)
    return gap, tpr_m, tpr_f
```

```
MLP(
  (fc1): Linear(in_features=14, out_features=64, bias=True)
  (fc2): Linear(in_features=64, out_features=64, bias=True)
  (fc3): Linear(in_features=64, out_features=1, bias=True)
  (relu): ReLU()
  (sigmoid): Sigmoid()
)
```

```python
# model = MLP(input_dim=X_train_scaled.shape[1])
# optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
# criterion = nn.BCELoss()
# threshold_default = 0.5

# history = {
#     "epoch": [],
#     "test_acc": [],
#     "tpr_male": [],
#     "tpr_female": [],
```

```
#     "gap_eqopp": []
# }

# y_test_flat = y_test_scaled.reshape(-1).astype(int)

# for epoch in range(1, 51):
#     model.train()
#     total_loss = 0.0

#     for xb, yb in train_loader:
#         optimizer.zero_grad()
#         out = model(xb)
#         loss = criterion(out, yb)
#         loss.backward()
#         optimizer.step()
#         total_loss += loss.item() * xb.size(0)

#     probs = predict_proba_full(model, X_test_scaled)
#     yhat = probs_to_pred(probs, threshold = threshold_default)

#     gap, tpr_m, tpr_f = equal_opportunity_gap(y_test_flat, yhat, sex_test)
#     test_a = acc(y_test_flat, yhat)

#     history["epoch"].append(epoch)
#     history["test_acc"].append(test_a)
#     history["tpr_male"].append(tpr_m)
#     history["tpr_female"].append(tpr_f)
#     history["gap_eqopp"].append(gap)

#     if epoch in [1, 20, 50]:
#         avg_loss = total_loss / len(train_loader.dataset)
#         print(f"epoch {epoch} | train_loss={avg_loss:.4f} | test_acc={test_a:.4f} | "
#               f"tpr_m={tpr_m:.4f} | tpr_f={tpr_f:.4f} | gap={gap:.4f}")

# print("\n final/last test accuracy : epoch 50 = ", round(history["test_acc"][-1], 5))
```

Baseline

```
def train_model(X_train_np, y_train_np,X_eval_np, y_eval_np,sex_eval,epochs=50,
                lr=0.01,batch_size=32,sample_weights=None,):

    model = MLP(input_dim=X_train_np.shape[1])

    if sample_weights is None:
        train_loader = make_loader(X_train_np, y_train_np, batch_size=batch_size, shuffle=True)
    else:
        train_loader = make_loader(
            X_train_np,
            y_train_np,
            batch_size=batch_size,
            shuffle=True,
            sample_weights=sample_weights,
        )

    optimizer = torch.optim.SGD(model.parameters(), lr=lr)

    if sample_weights is None:
        criterion = nn.BCELoss()
    else:
        criterion = nn.BCELoss(reduction='none')

    history = {
        'epoch': [],
        'test_acc': [],
        'tpr_male': [],
        'tpr_female': [],
        'gap_eqopp': [],
    }

    y_eval_flat = y_eval_np.reshape(-1).astype(int)

    for epoch in range(1, epochs + 1):
        model.train()
        total_loss = 0.0

        if sample_weights is None:
```

```
            for xb, yb in train_loader:
                optimizer.zero_grad()
                out = model(xb)
                loss = criterion(out, yb)
                loss.backward()
                optimizer.step()
                total_loss += loss.item() * xb.size(0)
        else:
            for xb, yb, wb in train_loader:
                optimizer.zero_grad()
                out = model(xb)
                per_sample_loss = criterion(out, yb)
                loss = (per_sample_loss * wb).mean()
                loss.backward()
                optimizer.step()
                total_loss += loss.item() * xb.size(0)

        probs = predict_proba_full(model, X_eval_np)
        yhat = probs_to_pred(probs, threshold=0.5)

        gap, tpr_m, tpr_f = equal_opportunity_gap(y_eval_flat, yhat, sex_eval)
        test_acc = acc(y_eval_flat, yhat)

        history['epoch'].append(epoch)
        history['test_acc'].append(test_acc)
        history['tpr_male'].append(tpr_m)
        history['tpr_female'].append(tpr_f)
        history['gap_eqopp'].append(gap)

        if epoch in [1, 20, 50]:
            avg_loss = total_loss / len(train_loader.dataset)
            print(
                f'epoch {epoch} | train_loss={avg_loss:.4f} | test_acc={test_acc:.4f} | '
                f'tpr_m={tpr_m:.4f} | tpr_f={tpr_f:.4f} | gap={gap:.4f}'
            )

    return model, history
```

```
torch.manual_seed(42)
b_model, b_history = train_model( #Baseline mdoels and history
    X_train_scaled,
    y_train_scaled,
    X_test_scaled,
    y_test_scaled,
    sex_test,
    epochs=50,
    lr=0.01,
    batch_size=32,
    seed=42,
    sample_weights=None,
)

baseline_gap = b_history['gap_eqopp'][-1]
baseline_acc = b_history['test_acc'][-1]
baseline_tpr_m = b_history['tpr_male'][-1]
baseline_tpr_f = b_history['tpr_female'][-1]
print(f'Baseline EqOpp gap: {baseline_gap:.4f}')
print(f'BAseline Test accuracy: {baseline_acc:.4f}')
print(f'TPR male: {baseline_tpr_m:.6f}, TPR female: {baseline_tpr_f:.6f}')
```

```
epoch 1 | train_loss=0.5064 | test_acc=0.7702 | tpr_m=0.0622 | tpr_f=0.0204 | gap=0.0418
epoch 20 | train_loss=0.3209 | test_acc=0.8429 | tpr_m=0.6447 | tpr_f=0.4870 | gap=0.1577
epoch 50 | train_loss=0.3130 | test_acc=0.8462 | tpr_m=0.6279 | tpr_f=0.4721 | gap=0.1558
Baseline EqOpp gap: 0.1558
BAseline Test accuracy: 0.8462
TPR male: 0.627907, TPR female: 0.472119
```

Double-click (or enter) to edit

```
Start coding or generate with AI.
```

Part 1: Oversampling (os)

```python
def oversample_female_positive(X_np, y_np, sex_np, seed=42):
    y_flat = y_np.reshape(-1).astype(int)

    male_pos_idx = np.where((sex_np == 1) & (y_flat == 1))[0]
    female_pos_idx = np.where((sex_np == 0) & (y_flat == 1))[0]

    before_counts = {
        'male_pos': int(len(male_pos_idx)),
        'female_pos': int(len(female_pos_idx)),
        'total': int(len(y_flat)),
    }

    if len(female_pos_idx) < len(male_pos_idx):
        rng = np.random.default_rng(seed)
        add_n = len(male_pos_idx) - len(female_pos_idx)
        sampled_idx = rng.choice(female_pos_idx, size=add_n, replace=True)

        X_new = np.vstack([X_np, X_np[sampled_idx]])
        y_new = np.vstack([y_np, y_np[sampled_idx]])
        sex_new = np.concatenate([sex_np, sex_np[sampled_idx]])
    else:
        X_new, y_new, sex_new = X_np, y_np, sex_np

    y_new_flat = y_new.reshape(-1).astype(int)
    after_counts = {
        'male_pos': int(np.sum((sex_new == 1) & (y_new_flat == 1))),
        'female_pos': int(np.sum((sex_new == 0) & (y_new_flat == 1))),
        'total': int(len(y_new_flat)),
    }

    return X_new, y_new, sex_new, before_counts, after_counts


X_over, y_over, sex_over, counts_before, counts_after = oversample_female_positive(
    X_train_scaled,
    y_train_scaled,
    sex_train,
    seed=42,
)

print('Before oversampling:', counts_before)
print('After oversampling :', counts_after)

```

```
Before oversampling: {'male_pos': 6881, 'female_pos': 1198, 'total': 33334}
After oversampling : {'male_pos': 6881, 'female_pos': 6881, 'total': 39017}
```

```python
def oversample(X_np, y_np, sex_np, seed=42):
    y_flat = y_np.reshape(-1).astype(int)
    male_pos_idx = np.where((sex_np == 1) & (y_flat == 1))[0]
    female_pos_idx = np.where((sex_np == 0) & (y_flat == 1))[0]

    before_os = {
        'male_pos': int(len(male_pos_idx)),
        'female_pos': int(len(female_pos_idx)),
        'total': int(len(y_flat)),
    }

    if len(female_pos_idx) < len(male_pos_idx):
        rng = np.random.default_rng(seed)
        add_n = len(male_pos_idx) - len(female_pos_idx)
        sampled_idx = rng.choice(female_pos_idx, size=add_n, replace=True)

        X_os = np.vstack([X_np, X_np[sampled_idx]])
        y_os = np.vstack([y_np, y_np[sampled_idx]])
        sex_os = np.concatenate([sex_np, sex_np[sampled_idx]])
    else:
        X_os, y_os, sex_os = X_np, y_np, sex_np

    y_os_flat = y_os.reshape(-1).astype(int)
    after_os = {
        'male_pos': int(np.sum((sex_os == 1) & (y_os_flat == 1))),
        'female_pos': int(np.sum((sex_os == 0) & (y_os_flat == 1))),
        'total': int(len(y_os_flat)),
    }

    return X_os, y_os, sex_os, before_os, after_os
```

```python
X_os, y_os, sex_os, count_before_os, count_after_os = oversample(
    X_train_scaled,
    y_train_scaled,
    sex_train,
    seed=42,
)

print('Before oversampling (as per Baseline):', count_before_os)
print('After oversampling :', count_after_os)

```

```
Before oversampling (as per Baseline): {'male_pos': 6881, 'female_pos': 1198, 'total': 33334}
After oversampling : {'male_pos': 6881, 'female_pos': 6881, 'total': 39017}
```

```python
assert counts_before['male_pos'] > 0 and counts_before['female_pos'] > 0
assert counts_after['female_pos'] == counts_after['male_pos']
```

```python
torch.manual_seed(42)
over_model, over_history = train_model(
    X_over,
    y_over,
    X_test_scaled,
    y_test_scaled,
    sex_test,
    epochs=50,
    lr=0.01,
    batch_size=32,
    seed=42,
    sample_weights=None,
)

over_gap = over_history['gap_eqopp'][-1]
over_acc = over_history['test_acc'][-1]
over_tpr_m = over_history['tpr_male'][-1]
over_tpr_f = over_history['tpr_female'][-1]

print(f'EqOpp gap: {over_gap:.4f}')
print(f'Test accuracy: {over_acc:.4f}')
print(f'TPR male: {over_tpr_m:.6f}, TPR female: {over_tpr_f:.6f}')
```

```
epoch 1 | train_loss=0.5738 | test_acc=0.7631 | tpr_m=0.3526 | tpr_f=0.7138 | gap=0.3612
epoch 20 | train_loss=0.3515 | test_acc=0.8303 | tpr_m=0.5684 | tpr_f=0.7751 | gap=0.2067
epoch 50 | train_loss=0.3361 | test_acc=0.8279 | tpr_m=0.5725 | tpr_f=0.8197 | gap=0.2472
EqOpp gap: 0.2472
Test accuracy: 0.8279
TPR male: 0.572503, TPR female: 0.819703
```

```
Start coding or generate with AI.
```

Q1 : part1

```python
print(f" Before \n(Male, >50K): {counts_before['male_pos']}, (Female, >50K) before: {counts_before['female_pos']}")
print(f"After \n(Male, >50K): {counts_after['male_pos']},(Female, >50K)  : {counts_after['female_pos']}")
```

```
 Before
(Male, >50K): 6881, (Female, >50K) before: 1198
After
(Male, >50K): 6881,(Female, >50K)  : 6881
```

Q1: part2

```python
print(f"Total training samples: \nBefore os = {counts_before['total']}, \nAfter os : {counts_after['total']},\nsample increas
#Os is Oversampling

```

```
Total training samples:
Before os = 33334,
After os : 39017,
sample increased by: 5683
```

Q1: part3

```
print(f"EqOp: {over_gap},\ntest acc: {over_acc}")
```

```
EqOp: 0.2471991822578431,
test acc: 0.8278854903058724
```

Q1: part4

```
print(f'Gap in EqOpp gap: {baseline_gap - over_gap}')
print(f'gap in Test accuracy: {baseline_acc - over_acc}')
```

```
Gap in EqOpp gap: -0.09141116462146376
gap in Test accuracy: 0.018268355847973705
```

Thus We can say after Oversampling(os), the gap in Equal opportunity increased and accuracy decreased.

This might happen as we duplicated the data which add less new information than real new data, which may have reduce the fairness, resulting in lower accuracy.

```
Start coding or generate with AI.
```

Part2: Re-weighting

w(group, label) = P(group) × P(label) / P(group, label)

```
# where P(group) is the probability of belonging to that group (e.g., P(Male)), P(label) is the probability of having that la

# Male, >50K ----> 1, 1
# Male, <=50K ---> 1, 0
# Female, >50K --> 0, 1
# Female, <=50K -> 0, 0
```

```
#Group-Label
def weight_label(y_np, sex_np):
    y_flat = y_np.reshape(-1).astype(int)
    g = sex_np.astype(int)

    p_g = {gi: np.mean(g == gi) for gi in [0, 1]}
    p_y = {yi: np.mean(y_flat == yi) for yi in [0, 1]}
    p_gy = {(gi, yi): np.mean((g == gi) & (y_flat == yi)) for gi in [0, 1] for yi in [0, 1]}

    w = {}
    for gi in [0, 1]:
        for yi in [0, 1]:
            w[(gi, yi)] = float((p_g[gi] * p_y[yi]) / p_gy[(gi, yi)])

    sample_weights = np.array([w[(int(gi), int(yi))] for gi, yi in zip(g, y_flat)], dtype=np.float32)
    return w, sample_weights
```

```
weights_map, sample_weights = weight_label(y_train_scaled, sex_train)
for k in sorted(weights_map.keys()):
    print(f'{k}: {weights_map[k]:.6f}')
```

```
(0, 0): 0.850441
(0, 1): 2.220939
(1, 0): 1.094519
(1, 1): 0.787431
```

```
torch.manual_seed(42)
rw_model, rw_history = train_model(
    X_train_scaled,
    y_train_scaled,
    X_test_scaled,
    y_test_scaled,
```

```
    sex_test,
    epochs=50,
    lr=0.01,
    batch_size=32,
    seed=42,
    sample_weights=sample_weights,
)

rw_gap = rw_history['gap_eqopp'][-1]
rw_acc = rw_history['test_acc'][-1]
rw_tpr_m = over_history['tpr_male'][-1]
rw_tpr_f = over_history['tpr_female'][-1]

print(f'EqOpp gap: {rw_gap:.4f}')
print(f'Test accuracy: {rw_acc:.4f}')
print(f'TPR male: {rw_tpr_m:.6f}, TPR female: {rw_tpr_f:.6f}')
```

```
epoch 1 | train_loss=0.5216 | test_acc=0.7652 | tpr_m=0.0304 | tpr_f=0.0446 | gap=0.0142
epoch 20 | train_loss=0.3265 | test_acc=0.8348 | tpr_m=0.5479 | tpr_f=0.6896 | gap=0.1417
epoch 50 | train_loss=0.3159 | test_acc=0.8402 | tpr_m=0.5554 | tpr_f=0.6450 | gap=0.0896
EqOpp gap: 0.0896
Test accuracy: 0.8402
TPR male: 0.572503, TPR female: 0.819703
```

Q2: part1

```
print(f"(Male, >50K) re-weight   : {weights_map[(1, 1)]},\n(Male, <=50K) re-weight : {weights_map[(1, 0)]}")
print(f"(Female, >50K) re-weight : {weights_map[(0, 1)]},\n(Female, <=50K) re-weight: {weights_map[(0, 0)]}")
```

```
(Male, >50K) re-weight   : 0.7874313840565217,
(Male, <=50K) re-weight : 1.0945192016999725
(Female, >50K) re-weight : 2.220938769872349,
(Female, <=50K) re-weight: 0.8504412427088882
```

Q2: part2

Female, >50k recevied the largest weight. As they were the most underprivileged group.

Q2: part3

```
print(f're-weighted, EqOpp gap: {rw_gap:.4f}')
print(f'Re-weighted Test accuracy: {rw_acc:.4f}')
```

```
re-weighted, EqOpp gap: 0.0896
Re-weighted Test accuracy: 0.8402
```

Q2: part4

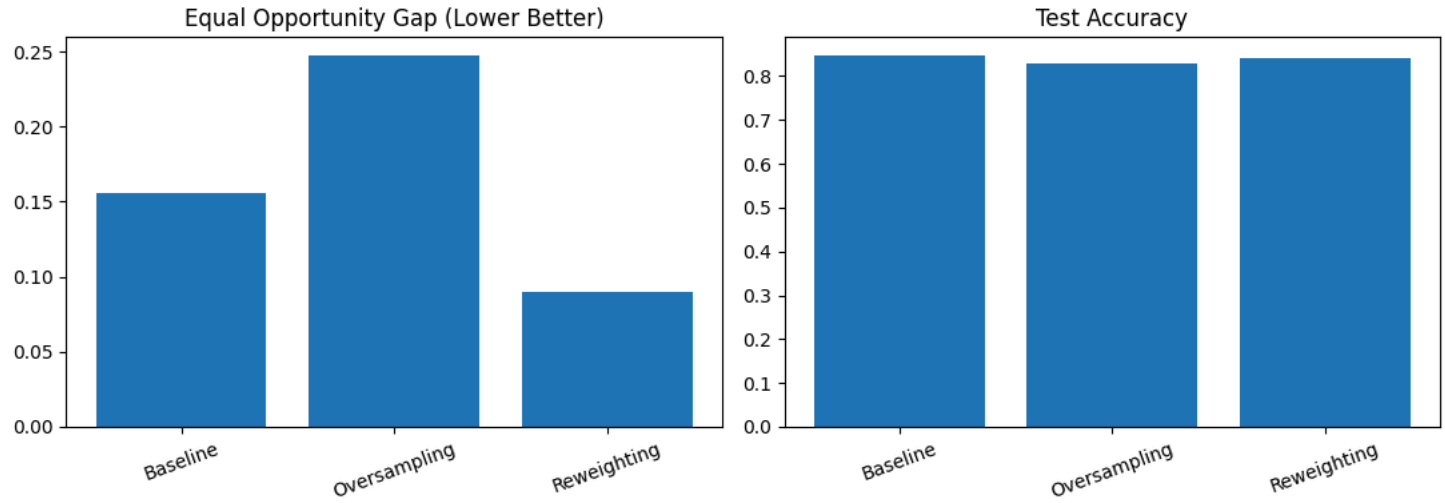
```
# EqOp: 0.14945731738098844,
# test acc: 0.8342257217847769
print("Baseline v/s os")
print(f'Gap in EqOpp gap: {baseline_gap - over_gap}')
print(f'gap in Test accuracy: {baseline_acc - over_acc}')
print("Baseline v/s RW")
print(f'Gap in EqOpp gap: {baseline_gap - rw_gap}')
print(f'gap in Test accuracy: {baseline_acc -rw_acc}')
```

```
Baseline v/s os
Gap in EqOpp gap: -0.09141116462146376
gap in Test accuracy: 0.018268355847973705
Baseline v/s RW
Gap in EqOpp gap: 0.06621016176851996
gap in Test accuracy: 0.005949464548190675
```

This behavior is expected from the reweighting objective. As it increases the training influence of underrepresented group label cells (here Female, >50K) and reduces influence of overrepresented cells (e.g., Male, >50K). As a result, the model improves recall balance across groups (smaller TPR difference), but the decision boundary is no longer optimized purely for unweighted overall accuracy, thus we saw a small accuracy dip. Therefore, reweighting moved the model toward a fairer TPR tradeoff at a minor accuracy cost, which is a common fairness-accuracy tradeoff.

```python
#Took AI help to quickly build this up.
results_df = pd.DataFrame([
    {
        'Method': 'Baseline',
        'Accuracy': baseline_acc,
        'TPR_male': baseline_tpr_m,
        'TPR_female': baseline_tpr_f,
        'EqOpp_Gap': baseline_gap,
    },
    {
        'Method': 'Oversampling',
        'Accuracy': over_acc,
        'TPR_male': over_tpr_m,
        'TPR_female': over_tpr_f,
        'EqOpp_Gap': over_gap,
    },
    {
        'Method': 'Reweighting',
        'Accuracy': rw_acc,
        'EqOpp_Gap': rw_gap,
    },
])

print(results_df.round(6))

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].bar(results_df['Method'], results_df['EqOpp_Gap'])
axes[0].set_title('Equal Opportunity Gap (Lower Better)')
axes[0].tick_params(axis='x', rotation=20)

axes[1].bar(results_df['Method'], results_df['Accuracy'])
axes[1].set_title('Test Accuracy')
axes[1].tick_params(axis='x', rotation=20)

plt.tight_layout()
plt.show()

```

```
         Method  Accuracy  TPR_male  TPR_female  EqOpp_Gap
0      Baseline  0.846154  0.627907    0.472119   0.155788
1  Oversampling  0.827885  0.572503    0.819703   0.247199
2   Reweighting  0.840204       NaN         NaN   0.089578
```




```
#HW3 completed
```

Start coding or generate with AI.

```
p
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```